TASK 1

1.1-1.4 Completed

1.5 The Open-Closed Principle means that software should be open for extension, but closed for modification. Factory Method is better because when a new Connector for example XMLConnector needs to be added, then it is simply a matter of extending the structure by adding an XMLFactory and XMLConnector as opposed to changing the existing code in the if-else block thus breaking the Open-Closed Principle.

1.6 Connector - This is the Abstract Product class.

CsvConnector - This is the Concrete Product class.

ConnectorFactory - This is the Creator class.

CsvFactory - This is the Concrete Creator class.

TASK 2

2.5 A shallow copy duplicates the object's member variables without duplicating any dynamically allocated resources and a deep copy duplicates the object's member variables including the duplication of its dynamically allocated resources. Clone returns a deep copy of the registered prototype. It must be a deep copy so that changes can be made to each object independently without affecting each other.

2.6

If two pipelines shared one prototype, that would be problematic because modifications could be made to the original or the clone and both objects would be changed. Returning a clone() returns a deep copy therefore changes can be made to the original and the clone independently. If the registry returned the stored pointer, then both pipelines would receive the exact same Transformation object.

TASK 3

3.5 run() is the template method: it specifies the algorithm order starting with connect(), extract(), transform() and finally load(). Keeping it non-virtual means subclasses cannot change that order or skip stages. If a subclass could override run(), it could call stages out of order (e.g. load before extract), skip transform(), or

never update stage. That would break the fixed lifecycle Kudu Freight relies on and make checkpoints meaningless, because stage numbers would no longer mean the same thing.

Subclasses may only vary the primitive steps (extract, load); the skeleton stays fixed in the base class.

3.6

run(): Template method (fixed skeleton; not virtual) connect(), transform(): Concrete steps (shared implementation in Pipeline) extract(), load(): Primitive operations (pure virtual; BatchPipeline / StreamingPipeline override)

Constructor, addStep(), destructor, createCheckpoint(), restore(): Supporting API (not part of the template-step classification)

- Factory Method (ConnectorFactory / createConnector) decides which connector object is created without naming Postgres/REST/CSV in Pipeline.

- Template Method (run) decides in what order the pipeline stages run. They do not overlap: one varies object creation, the other varies algorithm steps while keeping the stage sequence fixed.

TASK 4

4.4

A wide interface allows full access to memento internals (set/get all state).

Only the Originator (Pipeline) may use this when creating/restoring, so it can copy stage and records freely.

A narrow interface is limited. In this implementation, a safe API is exposed to others in this case getStage() and getRecords().

The Caretaker (CheckpointManager) and any outside code should use only this: store/return mementos without peeking into or mutating pipeline state arbitrarily.

4.5

a. After a successful stage (e.g. after transform(), stage = 3), the pipeline calls createCheckpoint(), which snapshots current stage and records into a RunCheckpoint.

b. That memento is passed to save() on CheckpointManager, which appends it to history.
c. load() then fails (e.g. at 02:00). The in-memory run is incomplete (stage may be mid-load or inconsistent).
d. On recovery, call undo() on the manager to get the last saved RunCheckpoint.
e. Call restore(checkpoint) on the pipeline so stage and records match the last good snapshot.
f. Resume from the next unfinished stage (here: retry load()), instead of redoing connect/extract/transform from scratch.

4.6

Pipeline: Originator

RunCheckpoint: Memento

CheckpointManager: Caretaker